GeoClaw-claude

Python 地理信息分析工具集

技 术 文 档

版本 v2.2.1 · 2025 年 3 月

UrbanComp Lab · https://urbancomp.net

MIT License · Copyright © 2025 UrbanComp Lab

1. 项目概述

1.1 背景与目标

GeoClaw-claude 是由 UrbanComp Lab（https://urbancomp.net）开发的轻量级 Python 地理信息分析工具集。项目参考 QGIS Processing Framework 的模块化架构，以 Python 原生生态为基础，面向城市地理空间数据分析场景，提供从数据获取、空间分析到可视化制图的完整工作流。

项目名称来源于其双核心定位：Geospatial Analysis（地理空间分析）+ Claude AI 驱动的智能操作系统。v2.x 起，工具集正式迈入自然语言驱动的智能 GIS 平台时代——用户可通过中文自然语言直接调用所有分析功能，并借助 Anthropic Claude API 实现 AI 辅助分析与 Skill 脚本编写。

1.2 主要特性

- **GIS 工作流:** 全流程覆盖

从 OSM/WFS/HTTP 数据下载，到矢量空间分析（缓冲区/叠加/最近邻/KDE）、路网计算（最短路径/等时圈/服务区）、栅格处理（DEM/坡度/重分类/分区统计），再到静态制图与交互地图，形成完整闭环。

- **NL 模块:** 自然语言 GIS 操作（v2.0 核心）

双引擎架构：AI 模式（Claude API）+ 规则模式（关键词+正则，完全离线）。30+ 操作类型，支持多步流水线，低置信度自动请求确认。

- **mobility 模块:** 人类移动性分析（v2.1）

整合 trackintel（ETH Zurich），提供 GPS 轨迹→停留点→出行段→重要地点五级数据层级处理链路，支持回转半径/跳跃距离/出行方式识别等指标计算，并可通过自然语言调用。

- **memory 模块:** 跨会话记忆系统（v1.1）

ShortTermMemory（TTL 缓存+操作日志）+ LongTermMemory（JSON 持久化+关键词检索）。会话结束自动 flush 复盘报告到长期记忆。

- **updater 模块:** 自我检测与自动更新（v1.2）

远程版本检测（GitHub raw API）、一键升级（git pull + pip install）、全面健康检测（模块/依赖/网络三维度）。

- **坐标转换:** 中国数据友好

内置 WGS84/GCJ-02/BD-09 坐标系完整互转（六方向），支持天地图 POI API，覆盖国内数据使用场景。

1.3 适用人群

用户类型	适用场景
科研人员 / 博士生	城市空间分析、GPS 轨迹研究、POI 分布、路网可达性、栅格地形分析
实验室本科生	学习 GIS 编程、复现论文实验、完成课程设计
外部合作者	快速上手武汉/其他城市数据分析，使用内置 Skill，自然语言操作无需学习 API
工程开发者	二次开发，自定义 Skill，集成到已有数据处理流程，CLI 命令管道化

2. 工程架构

2.1 整体架构

GeoClaw-claude 采用分层架构，自底向上分为数据层、分析层、智能层和接口层：

架构层	模块与职责
接口层	CLI（geoclaw-claude 命令族）+ Python API（直接 import 调用）+ GeoAgent（对话代理）
智能层	nl/（NLProcessor + NLExecutor）、memory/（短期+长期记忆）、skills/（AI Skill 系统）、updater（自检更新）
分析层	analysis/spatial_ops、analysis/network、analysis/raster_ops、analysis/mobility/（trackintel）、cartography/（制图）
数据层	io/（vector/osm/remote）、core/（GeoLayer/GeoClawProject）、utils/（coord_transform）、config（全局配置）

2.2 目录结构

```
GeoClaw_Claude/
├── geoclaw_claude/                  # 主包
│   ├── __init__.py                  # v2.2.1，公共 API 导出
│   ├── cli.py                       # Click CLI 入口（所有命令）
│   ├── config.py                    # 配置系统（JSON + 环境变量）
│   ├── updater.py                   # 自检与自动更新
│   ├── skill_manager.py             # Skill 加载、验证与运行
│   ├── core/
│   │   ├── layer.py                 # GeoLayer 核心图层类
│   │   └── project.py               # GeoClawProject 项目管理
│   ├── analysis/
│   │   ├── spatial_ops.py           # 矢量空间分析
│   │   ├── network.py               # 路网分析（osmnx 驱动）
│   │   ├── raster_ops.py            # 栅格分析（rasterio 驱动）
│   │   └── mobility/                # 人类移动性分析
│   │       ├── __init__.py          # 公共 API 导出
│   │       ├── core.py              # 数据层级生成（trackintel 封装）
│   │       ├── metrics.py           # 移动性指标计算
│   │       └── visualization.py     # 可视化（英文标签，暗色主题）
│   ├── nl/                          # 自然语言操作系统
│   │   ├── __init__.py
│   │   ├── processor.py             # NLProcessor（意图解析）
│   │   ├── executor.py              # NLExecutor（意图→GIS 执行）
│   │   └── agent.py                 # GeoAgent（多轮对话代理）
│   ├── memory/                      # 记忆系统
│   │   ├── short_term.py            # ShortTermMemory（TTL+操作日志）
│   │   ├── long_term.py             # LongTermMemory（JSON 持久化）
│   │   └── manager.py               # MemoryManager（全局单例）
│   ├── cartography/
│   │   ├── renderer.py              # 静态/交互制图
│   │   └── map_composer.py          # 多图层地图排版
```

```
│   ├── io/
│   │   ├── vector.py                # GeoJSON/SHP/CSV 读写
│   │   ├── osm.py                   # OSM/Overpass 下载
│   │   └── remote.py                # HTTP/WFS/天地图 API
│   ├── utils/
│   │   └── coord_transform.py       # WGS84↔GCJ-02↔BD-09
│   └── skills/
│       └── builtin/
│           └── hospital_coverage.py
├── data/
│   ├── wuhan/                       # 武汉示例数据（7 个 GeoJSON）
│   └── mobility/                    # GPS 轨迹测试数据
│       ├── wuhan_gps_tracks.csv     # 37,549 个轨迹点
│       ├── users_meta.csv
│       └── README.md
├── examples/
│   └── wuhan_mobility_demo.py       # 完整移动性分析 Demo
├── tests/
│   ├── test_memory.py               # 37 项
│   ├── test_updater.py              # 20 项
│   ├── test_nl.py                   # 20 项
│   └── test_mobility.py             # 20 项（共 97 项，全部通过）
├── docs/                            # 用户手册与技术文档
├── install.sh
├── setup.py
└── CHANGELOG.md
```

3. 核心数据模型

3.1 GeoLayer

GeoLayer 是 GeoClaw-claude 的核心数据结构，封装了 GeoDataFrame 并添加了元数据管理、CRS 感知和操作历史追踪：

```
from geoclaw_claude import GeoLayer
from geoclaw_claude.io.vector import load_vector

# 从文件加载
layer = load_vector("hospitals.geojson")
# layer 类型: GeoLayer
# layer.gdf       → GeoDataFrame（geopandas）
# layer.name      → 图层名称（自动从文件名提取）
# layer.crs       → 坐标参考系（pyproj.CRS）
# layer.meta      → 元数据字典
# layer.history   → 操作历史列表
```

3.2 移动性数据层级（trackintel）

移动性分析模块使用 trackintel 定义的五级 GeoDataFrame 层级，每级都有严格的列名规范：

层级名	索引名	必要列
positionfixes	id	user_id, tracked_at, geom（点几何）
staypoints	id	user_id, started_at, finished_at, geom（点几何）
triplegs	id	user_id, started_at, finished_at, geom（线几何），可选：mode
trips	id	user_id, started_at, finished_at, origin_staypoint_id, destination_staypoint_id
locations	id	user_id, center（点几何），可选：purpose（home/work）

💡 *使用 generate_full_hierarchy() 可一次性生成全部五级数据，内部自动处理层级间的数据传递与格式转换。*

4. 自然语言操作系统（nl/ 模块）

4.1 NLProcessor — 意图解析

NLProcessor 负责将自然语言输入转换为结构化的 ParsedIntent 对象，支持两种模式：

模式	触发条件	特点
AI 模式	config 中有 anthropic_api_key，或 --ai 参数	调用 Claude API，理解复杂语义，置信度准确
规则模式	无 API Key，或 --rule 参数（离线降级）	关键词+正则本地解析，无需网络，覆盖 30+ 标准操作

```
from geoclaw_claude.nl import NLProcessor

# 自动选择模式
p = NLProcessor()

# 强制规则模式（离线调试）
p = NLProcessor(use_ai=False)

intent = p.parse("对医院做1公里缓冲区")
# intent.action     → 'buffer'
# intent.params     → {'distance': 1000, 'unit': 'meters'}
# intent.targets    → ['hospitals']
# intent.confidence → 0.92
```

4.2 规则模式关键词体系

规则模式通过分层关键词字典实现意图映射，优先级从高到低：

操作类型	关键词示例	说明
mobility_load	"读入" "gps" "轨迹" "positionfix"	GPS 数据读入，高优先级避免被其他规则捕获
mobility_transport	"预测出行方式" "出行方式预测" "交通方式预测"	必须是精确词组，避免与 triplegs 冲突（v2.2.1 修复）
mobility_heatmap	"时间热力图" "活动热力图" "heatmap"	在 KDE 规则之前检测，避免误判为空间热力图
mobility_plot	"轨迹地图" "移动性地图"	在 render 规则之前检测
mobility_modal	"出行方式构成图" "出行构成图" "modal"	出行方式可视化，v2.2.1 补全词组
buffer	"缓冲区" "buffer" "米范围" "公里缓冲"	自动提取距离数值（支持中文数字）
kde	"核密度" "密度热力图" "kde"	在 mobility_heatmap 之后检测

4.3 NLExecutor — GIS 执行

NLExecutor 接收 ParsedIntent，映射到具体的 GIS 函数调用，管理图层上下文，并将操作写入 Memory：

```
from geoclaw_claude.nl import NLExecutor
from geoclaw_claude.nl.processor import NLProcessor

executor = NLExecutor(memory_manager=get_memory())

# 执行意图
result = executor.execute_intent(intent, layers={"hospitals": hospital_layer})
# result.success  → True/False
# result.result   → 执行结果（新图层/分析结果/图表）
# result.message  → 人类可读的结果描述
# result.duration → 耗时（毫秒）
```

💡 *executor 的 layers 参数是一个命名字典，GeoAgent 会在多轮对话中自动维护和更新这个字典，实现图层上下文跨轮传递。*

5. 人类移动性分析模块技术细节

5.1 模块架构

子模块	职责与主要函数
core.py	数据层级生成：read_positionfixes(), generate_staypoints(), generate_triplegs(), generate_trips(), generate_locations(), generate_full_hierarchy(), predict_transport_mode()
metrics.py	指标计算：radius_of_gyration(), jump_lengths(), modal_split(), tracking_quality(), identify_home_work(), mobility_summary()
visualization.py	可视化（英文标签，暗色主题）：plot_mobility_layers(), plot_activity_heatmap(), plot_modal_split(), plot_mobility_metrics()

5.2 核心函数详解

generate_full_hierarchy()

一键生成完整数据层级，是最推荐的使用方式：

```
hierarchy = generate_full_hierarchy(
    pfs,                        # positionfixes GeoDataFrame
    dist_threshold=80,          # 停留判定距离阈值（米），默认 80
    time_threshold=5,           # 停留判定时间阈值（分钟），默认 5
    gap_threshold=30,           # GPS 数据缺口容忍阈值（分钟），默认 30
    location_epsilon=120,       # 地点聚类半径（米），默认 120
    min_samples=2,              # DBSCAN 最小样本数，默认 2
    predict_mode=True,          # 是否预测出行方式，默认 True
)
# 返回字典，包含: positionfixes/staypoints/triplegs/trips/locations
```

predict_transport_mode()

基于速度特征对出行段进行交通方式分类：

mode 值	速度范围	说明
slow_mobility	< 15 km/h	步行、骑行等慢速出行
motorized_mobility	15 ~ 100 km/h	驾车、公交等机动化出行
fast_mobility	> 100 km/h	高铁、飞机等高速出行

mobility_summary()

计算综合指标摘要，返回包含以下键的字典：

键名	内容
n_users	用户数量
n_positionfixes	GPS 轨迹点总数

n_staypoints	停留点总数
n_triplegs	出行段总数
n_locations	重要地点总数
radius_of_gyration_m	{mean, median, max}（米），衡量用户活动范围大小
jump_length_m	{mean, median, p90}（米），单次出行距离分布
modal_split	{mode: share} 字典，出行方式占比
tracking_quality_mean	时间覆盖率均值（0~1），衡量数据完整性

5.3 trackintel 兼容性说明

⚠️ *以下是 trackintel 1.4.x API 与旧版本的差异，GeoClaw-claude 已在 core.py 中处理这些差异：*

- read_positionfixes_gpd() 参数名：crs_wgs84 → crs（v1.3+）
- generate_trips() 需要 staypoints 包含 is_activity 列（v1.3+），已在 generate_full_hierarchy() 中自动添加
- tracking_quality() 对 positionfixes 需将 tracked_at 列转换为 started_at/finished_at 格式
- radius_of_gyration() 返回列名为 radius_gyration（非 radius_of_gyration），已在 metrics.py 中做自动适配
- location_identifier() 参数顺序变化：method 改为位置参数而非关键字参数（v1.4+）

6. Memory 记忆系统技术细节

6.1 系统架构

Memory 系统采用三层架构：ShortTermMemory（内存）、LongTermMemory（磁盘）、MemoryManager（统一管理，全局单例）。

6.2 ShortTermMemory 实现

- **内存:** 存储介质

使用 Python 字典存储，RLock 保证线程安全。

- **时间过期:** TTL 机制

每个缓存条目支持独立 TTL（Time To Live），默认 3600 秒，超时自动失效。

- **history:** 操作日志

每次 log_op() 调用追加到操作历史列表，记录操作类型、参数、时间戳和耗时。

- **context:** 上下文变量

通过 set_context() / get_context() 管理会话内全局变量（如当前城市、CRS 等）。

6.3 LongTermMemory 实现

- **~/.geoclaw_claude/memory/:** 存储路径

每个记忆条目独立存储为 {entry_id}.json 文件，支持并发访问。

- **关键词+标签:** 检索机制

recall() 函数对 title/content/tags 字段进行关键词全文检索，按 importance 和 accessed_at 排序。

- **decay:** 重要性衰减

长期未访问的记忆 importance 会缓慢衰减（compact() 触发），防止存储空间无限增长。

6.4 会话复盘数据结构

end_session() 自动生成的复盘记录包含以下字段：

```
{
  "session_id":   "wuhan_hospital_analysis",
  "started_at":   "2025-03-07T10:00:00",
  "ended_at":     "2025-03-07T10:25:00",
  "duration_min": 25.0,
  "operations":   [                          // 操作序列
    {"op": "load",   "params": "hospitals.geojson", "duration_ms": 120},
    {"op": "buffer", "params": "1km",               "duration_ms": 430},
    {"op": "kde",    "params": "bandwidth=0.05",    "duration_ms": 280},
  ],
  "success_count": 3,
  "fail_count":    0,
  "cached_layers": ["hospitals", "buf_hospitals"]  // 缓存图层列表
}
```

7. 空间分析 API 参考

7.1 矢量空间分析（spatial_ops.py）

函数签名	参数说明
`buffer(layer, distance, unit='meters')`	distance: 数值；unit: meters/km/degrees。内部自动处理 CRS（UTM 投影精确计算）
`clip(layer, mask_layer)`	用 mask_layer 裁剪 layer，自动对齐 CRS
`nearest_neighbor(layer, target)`	计算 layer 中每个要素到 target 的最近邻距离（米），返回含 nearest_id 和 nearest_dist_m 新列的 GeoLayer
`kde(layer, bandwidth, grid_size)`	KDE 核密度估计，bandwidth：带宽（度），grid_size：网格数，返回 numpy 二维数组
`overlay(layer1, layer2, how)`	how: intersection/union/difference/symmetric_difference
`zonal_stats_vector(layer, zones, stats)`	矢量分区统计，stats 列表支持: count/sum/mean/min/max/std

7.2 路网分析（network.py）

函数签名	参数说明
`build_network(bbox, network_type)`	bbox: (min_lon, min_lat, max_lon, max_lat)；network_type: drive/walk/bike/all；基于 osmnx 下载 OSM 路网
`shortest_path(G, origin, destination, weight)`	origin/destination: (lon, lat)；weight: length/travel_time；返回路径 GeoDataFrame + 距离/时间
`isochrone(G, center, minutes)`	center: (lon, lat)；minutes: 列表，如 [5,10,15]；返回等时多边形 GeoDataFrame
`service_area(G, sources, cutoff)`	sources: 多个 (lon,lat) 列表；cutoff: 距离/时间阈值；批量计算服务区并合并

7.3 栅格分析（raster_ops.py）

函数签名	参数说明
`load_raster(path)`	返回 (data: ndarray, transform: Affine, crs: CRS) 元组
`slope(dem, units)`	计算坡度；units: degrees/percent_rise；使用 Horn 算法
`aspect(dem)`	计算坡向（0°=北，顺时针）
`reclassify(raster, rules)`	rules: [(low, high, new_val), ...] 列表；支持连续值区间重分类
`zonal_stats(raster, zones, stats)`	分区统计，zones: GeoLayer；stats: mean/max/min/std/count 列表

`clip_raster(raster, mask_layer)`	按矢量图层裁剪栅格（含 nodata 处理）
`resample(raster, target_resolution)`	重采样至指定分辨率（米），默认双线性内插

8. Skill 脚本系统技术细节

8.1 Skill 规范

一个合规的 Skill 脚本必须包含 SKILL_META 字典和 run(ctx) 函数：

```
# my_skill.py

SKILL_META = {
    "name":        "my_analysis",          # 唯一标识符（小写+下划线）
    "version":     "1.0.0",                # 语义化版本号
    "author":      "YourName",
    "description": "我的分析脚本",
    "tags":        ["spatial", "hospital"],  # 可选标签
    "params": {                              # 参数定义
        "radius_km": {
            "type":    "float",
            "default": 5.0,
            "desc":    "服务半径（km）",
        },
    }
}

def run(ctx):
    # ctx.get_layer(name)      → 获取输入图层
    # ctx.param(name, default) → 获取运行参数
    # ctx.buffer(layer, dist)  → 调用内置空间分析
    # ctx.ask_ai(prompt)       → 调用 Claude AI（需 API Key）
    # ctx.info(message)        → 输出信息
    # ctx.result(output=layer) → 返回结果
    ...
```

8.2 SkillContext 接口

方法	说明
`ctx.get_layer(name)`	从当前图层上下文中获取指定名称的 GeoLayer
`ctx.param(name, default=None)`	获取 CLI 或 API 传入的参数值，若未提供则返回 default
`ctx.buffer(layer, dist_m)`	调用 spatial_ops.buffer()（单位：米）
`ctx.ask_ai(prompt)`	调用 Claude API，返回 AI 文字响应（需配置 anthropic_api_key）
`ctx.info(message)`	向用户输出信息（CLI 模式打印，API 模式追加到日志）
`ctx.result(output, message)`	返回 Skill 执行结果；output 可以是 GeoLayer/DataFrame/dict 等

9. 测试套件

GeoClaw-claude 维护 97 个单元测试，分布在 4 个测试文件中，全部通过（97/97）：

测试文件	测试数	覆盖范围
tests/test_memory.py	37 项	ShortTermMemory（TTL/线程安全/操作日志）、LongTermMemory（CRUD/检索/衰减）、MemoryManager（会话/flush）
tests/test_updater.py	20 项	check()版本检测、update()更新流程、self_check()健康报告、changelog_diff()版本差异
tests/test_nl.py	20 项	NLProcessor 规则模式（30+ 操作类型）、NLExecutor 执行链路、GeoAgent 多轮对话
tests/test_mobility.py	20 项	read_positionfixes、generate_full_hierarchy、mobility_summary、identify_home_work、plot_* 可视化函数、版本号验证

运行全部测试：

```
# 运行全部测试套件
python -m pytest tests/ -v

# 运行指定模块
python tests/test_mobility.py
python tests/test_nl.py
```

⚠️ *更新后建议运行 geoclaw-claude update --test 自动触发全量测试验证。*

10. 依赖清单与配置参考

10.1 完整依赖清单

包名	版本	类型	用途
geopandas	≥ 1.0	必需	矢量数据处理（GeoDataFrame）
shapely	≥ 2.0	必需	几何计算（缓冲/叠加/裁剪）
pyproj	≥ 3.5	必需	CRS 投影与坐标系转换
rasterio	≥ 1.3	必需	栅格数据 I/O 与处理
networkx	≥ 3.0	必需	路网图结构与算法
osmnx	≥ 2.0	必需	OSM 路网下载与处理
trackintel	≥ 1.4.2	必需	GPS 轨迹人类移动性分析
scipy	≥ 1.10	必需	KDE 核密度估计与统计分析
numpy	≥ 1.24	必需	数值计算基础库
pandas	≥ 2.0	必需	表格数据处理
matplotlib	≥ 3.7	必需	静态地图与图表生成
folium	≥ 0.14	必需	交互式 Leaflet 地图
click	≥ 8.0	必需	CLI 命令行框架
mapclassify	≥ 2.6	必需	专题地图分级着色
anthropic	≥ 0.20	可选	Claude AI API（NL AI 模式 + Skill AI）
contextily	≥ 1.4	可选	地图底图（Tiles）叠加

10.2 config.json 配置字段参考

字段名	默认值	说明
anthropic_api_key	""	Anthropic API Key（用于 NL AI 模式和 Skill AI 调用）
data_dir	"~/gis_data"	本地 GIS 数据存储根目录
default_crs	"EPSG:4326"	新建图层的默认坐标参考系
nl_mode	"auto"	auto（有 Key 用 AI/无 Key 用规则）/ ai / rule
nl_confidence_threshold	0.60	低于此置信度时 GeoAgent 主动请求确认
memory_dir	"~/.geoclaw_claude/memory/"	长期记忆存储目录
memory_max_entries	1000	长期记忆最大条目数（超出后触发 compact）

update_check_interval_days	7	自动检测新版本的间隔天数（0 表示禁用）

GeoClaw-claude v2.2.1 · 技术文档 · MIT License · © 2025 UrbanComp Lab · https://urbancomp.net